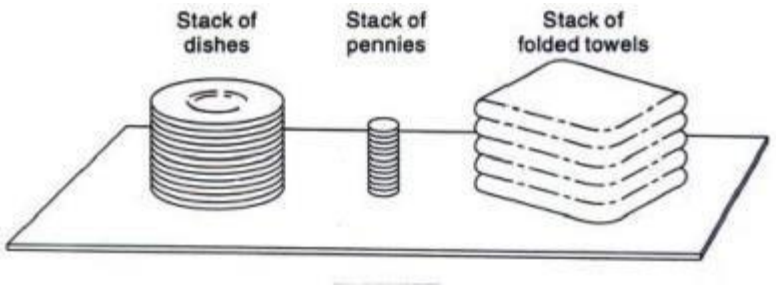
Stacks

STACK:

An stack is alist of elements in which an element may be inserted or deleted only at one end called the TOP of stack.

‘Push’ is the term used to insert an element into the stack

‘Pop’ is the term used to delete an element from the stack.



PUSH ALGORITHM (INSERTION IN STACK):

PUSH_LINKSTACK(INFO, LINK, TOP, AVAIL, ITEM)
This procedure pushes an ITEM into a linked stack

1. [Available space?] If AVAIL = NULL, then Write
   OVERFLOW and Exit
2. [Remove first node from AVAIL list]
   Set NEW := AVAIL and AVAIL := LINK[AVAIL].
3. Set INFO[NEW] := ITEM [Copies ITEM into new node]
4. Set LINK[NEW] := TOP [New node points to the original top node in the stack]
5. Set TOP = NEW [Reset TOP to point to the new node at the top of the stack]
6. Exit.

POP ALGORITHM (DELETION IN STACK)

POP_LINKSTACK(INFO, LINK, TOP, AVAIL, ITEM)
This procedure deletes the top element of a linked stack and assigns it to the variable ITEM

1. [Stack has an item to be removed?]
   IF TOP = NULL then Write: UNDERFLOW and Exit.
2. Set ITEM := INFO[TOP] [Copies the top element of stack into ITEM]
3. Set TEMP := TOP and TOP = LINK[TOP]
   [Remember the old value of the TOP pointer in TEMP
   and reset TOP to point to the next element in the stack]
4. [Return deleted node to the AVAIL list]
   Set LINK[TEMP] = AVAIL and AVAIL = TEMP.
5. Exit.

ARITHMETIC EXPRESSIONS; POLISH NOTATIONS:

- The Binary operations in arithmetic expressions have different levels of Precedence.

- The following three levels of Precedence for the usual five binary operations are:

- Highest : Exponentiation (^)
  Next Highest : Multiplication (*) and Division (/)
  Lowest: Addition (+) and Subtraction (-)

Example:

Evaluate the following parenthesis free arithmetic expression:

2^3+5*2^2-12/6

Solution:
First solve the exponent:

=8+5*4-12/6

Next solve the multiplication and division:

=8+20-2

Next solve addition and subtraction:
=26

NOTATIONS:

- **INFIX NOTATION:**

The operator symbol is placed between operands

A+B C-D E*F G/H

- **POLISH NOTATION or PREFIX NOTATION:**

The operator symbol is placed before its operands:

+AB -CD *EF /GH

- **REVERSE POLISH NOTATION or POSTFIX NOTATION:**

The operator symbol is placed after its operand.

AB+ CD- EF* GH/

EXAMPLE:

Convert the following infix notation into Prefix and Postfix notation.

(A+B) * C

SOLUTION:

PREFIX:

+AB * C

*+ABC

POSTFIX:

AB+ * C

AB+C*

EVALUATION OF POSTFIX EXPRESSION WITH STACK:

Consider the following arithmetic expression written in postfix notation:

5,6,2,+,*,12,4,/,-

To evaluate the above expression using stack:

Symbol Scanned	STACK
5	5
6	5,6
2	5,6,2
+	5, 8
*	40
12	40,12
4	40,12,4
/	40,3
-	37

QUICK SORT ALGORITHM; APPLICATION OF STACK

Quicksort is an algorithm of the divide-and-conquer type. That is, the problem of sorting a set is reduced to the problem of sorting two smaller sets. We illustrate this "reduction step" by means of a specific example.

Suppose A is the following list of 12 numbers:

(44) 33, 11, 55, 77, 90, 40, 60, 99, 22, 88, (66)

The reduction step of the quicksort algorithm finds the final position of one of the numbers; in this illustration, we use the first number, 44. This is accomplished as follows. Beginning with the last number, 66, scan the list from right to left, comparing each number with 44 and stopping at the first number less than 44. The number is 22. Interchange 44 and 22 to obtain the list

(22) 33, 11, 55, 77, 90, 40, 60, 99, (44) 88, 66

(Observe that the numbers 88 and 66 to the right of 44 are each greater than 44.) Beginning with 22, next scan the list in the opposite direction, from left to right, comparing each number with 44 and stopping at the first number greater than 44. The number is 55. Interchange 44 and 55 to obtain the list

22, 33, 11, (44) 77, 90, 40, 60, 99, (55) 88, 66

(Observe that the numbers 22, 33 and 11 to the left of 44 are each less than 44.) Beginning this time with 55, now scan the list in the original direction, from right to left, until meeting the first number less than 44. It is 40. Interchange 44 and 40 to obtain the list

22, 33, 11, (40) 77, 90, (44) 60, 99, 55, 88, 66

(Again, the numbers to the right of 44 are each greater than 44.) Beginning with 40, scan the list from left to right. The first number greater than 44 is 77. Interchange 44 and 77 to obtain the list

22, 33, 11, (40) (44) 90, (77) 60, 99, 55, 88, 66

(Again, the numbers to the left of 44 are each less than 44.) Beginning with 77, scan the list from right to left seeking a number less than 44. We do not meet such a number before meeting 44. This means all numbers have been scanned and compared with 44. Furthermore, all numbers less than 44 now form the sublist of numbers to the left of 44, and all numbers greater than 44 now form the sublist of numbers to the right of 44, as shown below:

22, 33, 11, 40, (44) 90, 77, 60, 99, 55, 88, 66

First sublist Second sublist

Thus 44 is correctly placed in its final position, and the task of sorting the original list A has now been reduced to the task of sorting each of the above sublists.

The above reduction step is repeated with each sublist containing 2 or more elements. Since we can process only one sublist at a time, we must be able to keep track of some sublists for future processing. This is accomplished by using two stacks, called LOWER and UPPER, to temporarily "hold" such sublists. That is, the addresses of the first and last elements of each sublist, called its *boundary values*, are pushed onto the stacks LOWER and UPPER, respectively; and the reduction step is applied to a sublist only after its boundary values are removed from the stacks. The following example illustrates the way the stacks LOWER and UPPER are used.

QUICK SORT ALGORITHM:

: QUICK(A, N, BEG, END, LOC)

Here A is an array with N elements. Parameters BEG and END contain the boundary values of the sublist of A to which this procedure applies. LOC keeps track of the position of the first element A[BEG] of the sublist during the procedure. The local variables LEFT and RIGHT will contain the boundary values of the list of elements that have not been scanned.

1. [Initialize.] Set LEFT := BEG, RIGHT := END and LOC := BEG.
2. [Scan from right to left.]
   (a) Repeat while A[LOC] ≤ A[RIGHT] and LOC ≠ RIGHT:
       RIGHT := RIGHT - 1.
   [End of loop.]
   (b) If LOC = RIGHT, then: Return.
   (c) If A[LOC] > A[RIGHT], then:
       (i) [Interchange A[LOC] and A[RIGHT].]
           TEMP := A[LOC], A[LOC] := A[RIGHT],
           A[RIGHT] := TEMP.
       (ii) Set LOC := RIGHT.
       (iii) Go to Step 3.
   [End of If structure.]
3. [Scan from left to right.]
   (a) Repeat while A[LEFT] ≤ A[LOC] and LEFT ≠ LOC:
       LEFT := LEFT + 1.
   [End of loop.]
   (b) If LOC = LEFT, then: Return.
   (c) If A[LEFT] > A[LOC], then
       (i) [Interchange A[LEFT] and A[LOC].]
           TEMP := A[LOC], A[LOC] := A[LEFT],
           A[LEFT] := TEMP.
       (ii) Set LOC := LEFT.
       (iii) Go to Step 2.
   [End of If structure.]

EXERCISE:

1) Consider the following stack of characters where stack is allocated N=8 memory cells:

STACK: A,C, D, F,K,____, ____, _____

Perform following operations on Stack.

a) POP (STACK, ITEM)
b) POP(STACK, ITEM)
c) PUSH(STACK, L)
d) PUSH (STACK, P)
e) POP (STACK ,ITEM)
f) PUSH (STACK, R)
g) PUSH (STACK,S)
h) POP (STACK, ITEM)

2) Translate the following expressions into their equivalent postfix and prefix expressions:

a) (A-B)*(D/E)
b) (A+B^D)/(E-F)+G

3) Evaluate the following postfix expression:

12,7,3,-,/,2,1,5,+,*,+

4) Suppose S is the list of 14 alphabetic characters which are sorted alphabetically:

D A T A S T R U C T U R E S

Perform quick sort to find the final position of first character D.